

Punto 2.5 - Relaciones muchos a muchos

Ejercicio guiado: modelar la relación muchos-a-muchos entre `OrdenFabricacion` y `Aleacion` mediante una entidad intermedia explícita `OrdenAleacion`. La relación no es una simple tabla puente: registra `FechaAsignacion`, `CantidadUtilizada` y `EstadoRelacion`, por lo que debe tratarse como una entidad con significado propio.

Contexto del proyecto: 2.5 continúa directamente desde 2.4. El modelo ya contiene la relación 1:N entre órdenes y planchas y las relaciones 1:1 de detalle y certificado. Hasta este punto `OrdenFabricacion` y `Aleacion` existen como entidades independientes, pero **no existe una relación muchos-a-muchos implícita previa**. En 2.5 se crea por primera vez la entidad `OrdenAleacion`, sus dos relaciones uno-a-muchos y la tabla `OrdenesAleaciones`.

Objetivos de aprendizaje

- Entender cuándo una relación muchos-a-muchos necesita una entidad intermedia explícita.
- Crear `OrdenAleacion` sin una propiedad `Id` artificial.
- Definir una clave primaria compuesta con `OrdenFabricacionId` y `AleacionId`.
- Añadir datos propios de la relación: fecha, cantidad y estado.
- Configurar las colecciones `OrdenesAleaciones` en ambos extremos.
- Configurar las dos relaciones uno-a-muchos que forman el muchos-a-muchos.
- Diferenciar `DeleteBehavior.Cascade` y `DeleteBehavior.Restrict`.
- Generar y leer la migración real `M2_2_5`.
- Comprobar la tabla intermedia y sus restricciones en SQL Server.
- Persistir una relación mediante el grafo de entidades.
- Recuperar la aleación con `Include` y `ThenInclude`.
- Entender cómo la PK compuesta impide duplicar el mismo par orden/aleación.
- Resolver el reto con dos aleaciones y cantidades diferentes.

Paso 1: Abrir el estado autónomo de 2.5

```
cd M02/PROYECTO/2.5
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

El checkpoint contiene el estado completo de 2.5. Si estás reproduciendo el laboratorio de forma incremental, parte de una copia de 2.4 y aplica los cambios siguientes.

Paso 2: Revisar el estado heredado de `OrdenFabricacion` y `Aleacion`

Antes de añadir la relación, recuerda qué existe en 2.4:

- `OrdenFabricacion` ya tiene `Planchas`, `Detalle` y `Certificado`.
- `Aleacion` ya tiene sus propiedades escalares de nombre, código y composición.
- No hay una colección directa `List<Aleacion>` en la orden.
- No hay una colección directa `List<OrdenFabricacion>` en la aleación.
- No existe todavía una tabla implícita `AleacionOrdenFabricacion`.

Esto es importante porque la migración real de 2.5 **no elimina una tabla puente anterior**. Crea `OrdenesAleaciones` directamente a partir del modelo de 2.4.

Paso 3: Crear la entidad intermedia `OrdenAleacion`

La entidad final del punto es:

```
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    public decimal CantidadUtilizada { get; set; }
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}
```

Interpreta sus propiedades:

1. `OrdenFabricacionId` identifica la orden y formará parte de la PK.
2. `AleacionId` identifica la aleación y completa la PK compuesta.
3. `FechaAsignacion` registra cuándo se creó la relación.
4. `CantidadUtilizada` pertenece a la relación, no a la orden ni a la aleación por separado.
5. `EstadoRelacion` permite expresar el estado propio de la asignación.
6. `Orden` y `Aleacion` son las navegaciones hacia los dos extremos.

No añadas una propiedad `Id`. El identificador natural de esta entidad es el par (`OrdenFabricacionId`, `AleacionId`).

Paso 4: Añadir las colecciones de entidades intermedias

`OrdenFabricacion` incorpora:

```
public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
```

y `Aleacion` incorpora la misma colección desde su extremo:

```
public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
```

La orden no almacena directamente una lista de `Aleacion`. Almacena las asignaciones, porque cada asignación posee `CantidadUtilizada`, `FechaAsignacion` y `EstadoRelacion`.

Paso 5: Exponer OrdenAleacion desde el DbContext

Añade:

```
public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();
```

El `DbSet` hace visible la entidad intermedia para consultas, inserciones y operaciones directas sobre la relación.

Paso 6: Configurar tabla, clave y propiedades de la relación

En `OnModelCreating`:

```
modelBuilder.Entity<OrdenAleacion>(entity =>
{
    entity.ToTable("OrdenesAleaciones");
    entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });

    entity.Property(x => x.FechaAsignacion)
        .HasDefaultValueSql("GETDATE()");

    entity.Property(x => x.CantidadUtilizada)
        .HasPrecision(18, 3);

    entity.Property(x => x.EstadoRelacion)
        .IsRequired()
        .HasMaxLength(20)
        .HasDefaultValue("Activa");
});
```

La clave:

```
entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
```

impide dos filas con exactamente el mismo par orden/aleación.

`CantidadUtilizada` usa precisión `18,3`, coherente con cantidades decimales que necesitan tres posiciones. `EstadoRelacion` queda requerido, limitado a 20 caracteres y con valor por defecto `"Activa"`.

▮ Paso 7: Configurar las dos relaciones uno-a-muchos

Dentro del mismo bloque se configuran los dos extremos:

```
entity.HasOne(x => x.Orden)
    .WithMany(o => o.OrdenesAleaciones)
    .HasForeignKey(x => x.OrdenFabricacionId)
    .OnDelete(DeleteBehavior.Cascade);

entity.HasOne(x => x.Aleacion)
    .WithMany(a => a.OrdenesAleaciones)
    .HasForeignKey(x => x.AleacionId)
    .OnDelete(DeleteBehavior.Restrict);
});
```

La relación muchos-a-muchos explícita se compone realmente de:

```
OrdenFabricacion 1 ---- N OrdenAleacion N ---- 1 Aleacion
```

Hacia `OrdenFabricacion` se usa `Cascade`. Si una orden se elimina físicamente, sus filas de asignación dejan de tener sentido y pueden eliminarse.

Hacia `Aleacion` se usa `Restrict`. Una aleación referenciada no debe desaparecer dejando asignaciones inválidas. Antes habría que eliminar o reasignar conscientemente esas relaciones.

No configures `Cascade` indiscriminadamente en ambos extremos.

▮ Paso 8: Compilar y generar la migración M2_2_5

Si estás construyendo 2.5 desde una copia de 2.4:

```
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations add M2_2_5
```

En el checkpoint final la migración ya existe:

```
Migrations/20260927204745_M2_2_5.cs
```

No vuelvas a generar otra migración con el mismo nombre dentro de la solución entregada.

Paso 9: Revisar la migración real

La migración crea directamente `OrdenesAleaciones`:

```
migrationBuilder.CreateTable(
    name: "OrdenesAleaciones",
    columns: table => new
    {
        OrdenFabricacionId = table.Column<int>(type: "int", nullable: false),
        AleacionId = table.Column<int>(type: "int", nullable: false),
        FechaAsignacion = table.Column<DateTime>(
            type: "datetime2",
            nullable: false,
            defaultValueSql: "GETDATE()"),
        CantidadUtilizada = table.Column<decimal>(
            type: "decimal(18,3)",
            precision: 18,
            scale: 3,
            nullable: false),
        EstadoRelacion = table.Column<string>(
            type: "nvarchar(20)",
            maxLength: 20,
            nullable: false,
            defaultValue: "Activa")
    },
    constraints: table =>
    {
        table.PrimaryKey(
            "PK_OrdenesAleaciones",
            x => new { x.OrdenFabricacionId, x.AleacionId });
    });
```

La migración **no contiene** `DropTable("AleacionOrdenFabricacion")`, porque 2.4 no tenía una tabla many-to-many implícita. Esperar ese `DropTable` sería describir un estado anterior que este curso no tiene.

Paso 10: Revisar las claves foráneas y el índice

La misma migración crea:

```
table.ForeignKey(
    name: "FK_OrdenesAleaciones_Aleaciones_AleacionId",
    column: x => x.AleacionId,
    principalTable: "Aleaciones",
    principalColumn: "Id",
    onDelete: ReferentialAction.Restrict);

table.ForeignKey(
    name: "FK_OrdenesAleaciones_OrdenesFabricacion_OrdenFabricacionId",
    column: x => x.OrdenFabricacionId,
    principalTable: "OrdenesFabricacion",
    principalColumn: "Id",
    onDelete: ReferentialAction.Cascade);
```

y un índice auxiliar sobre `AleacionId`:

```
migrationBuilder.CreateIndex(
    name: "IX_OrdenesAleaciones_AleacionId",
    table: "OrdenesAleaciones",
    column: "AleacionId");
```

La PK compuesta ya comienza por `OrdenFabricacionId`; el índice adicional facilita la navegación y la FK por `AleacionId`.

Paso 11: Aplicar la migración y comprobar SQL Server

Ejecuta:

```
dotnet ef database update
dotnet ef migrations list
```

En SQL Server Object Explorer comprueba `dbo.OrdenesAleaciones` y verifica:

- `OrdenFabricacionId` no nullable;
- `AleacionId` no nullable;
- `FechaAsignacion`;
- `CantidadUtilizada decimal(18,3)`;
- `EstadoRelacion nvarchar(20)`;
- PK compuesta (`OrdenFabricacionId`, `AleacionId`);
- FK a `OrdenesFabricacion` con eliminación en cascada;
- FK a `Aleaciones` con eliminación restrictiva;
- índice `IX_OrdenesAleaciones_AleacionId`.

El historial sigue gestionado mediante Migrations. No uses `EnsureCreated()`.

Paso 12: Crear una orden con una asignación de aleación

El checkpoint crea primero la orden y una aleación y las conecta mediante la entidad intermedia:

```
var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};

orden.OrdenesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
    EstadoRelacion = "Activa"
});

context.OrdenesFabricacion.Add(orden);
context.SaveChanges();
```

No es necesario guardar primero la aleación para obtener manualmente su Id. EF Core conoce el grafo y puede ordenar las inserciones, generar las claves y propagar las FKs.

La fila intermedia queda identificada por el Id de la orden y el Id de la aleación.

Paso 13: Recuperar el grafo con Include y ThenInclude

La comprobación funcional es:

```
var cargada = context.OrdenesFabricacion
    .Include(o => o.OrdenesAleaciones)
    .ThenInclude(x => x.Aleacion)
    .Single(o => o.NumeroOrden == "OF-M2-0001");

global::System.Console.WriteLine(
    $"2.5 OK | Aleaciones: {cargada.OrdenesAleaciones.Count} | " +
    $"Primera: {cargada.OrdenesAleaciones[0].Aleacion.Codigo}");
```

La salida esperada contiene:

2.5 OK | Aleaciones: 1 | Primera: A1045

`Include(o => o.OrdenesAleaciones)` carga las filas de la entidad intermedia. `ThenInclude(x => x.Aleacion)` continúa desde cada asignación hasta la entidad `Aleacion`.

Si omites `ThenInclude`, no debes asumir que `OrdenAleacion.Aleacion` estará cargada en una consulta nueva.

| Paso 14: Comprender actualización, eliminación y restricciones

La entidad intermedia permite tratar la asignación como un registro normal.

Cambiar la cantidad significa modificar `OrdenAleacion.CantidadUtilizada` y guardar.

Eliminar una asignación significa eliminar la fila concreta de `OrdenesAleaciones`; no implica borrar ni la orden ni la aleación.

Eliminar la orden elimina en cascada sus filas `OrdenAleacion`.

Eliminar la aleación mientras está referenciada debe ser rechazado por `Restrict`.

Insertar otra vez el mismo par de claves debe fallar por la PK compuesta.

Estas reglas derivan directamente de la configuración y de las restricciones visibles en la migración. No se sustituyen por lógica manual en memoria.

| Paso 15: Resolver el reto con dos aleaciones y cantidades diferentes

Reto: asignar dos aleaciones distintas a una nueva orden, cada una con su propia `CantidadUtilizada`, y recuperarlas mediante `Include` + `ThenInclude`.

El checkpoint contiene el bloque pedagógico `RETO 2.5 - DOS ALEACIONES CON THENINCLUDE`. Crea dos aleaciones:

```
var aleacionReto1 = new Aleacion
{
    Nombre = "AISI 1018",
   Codigo = "A1018",
    PorcentajeCarbono = 0.18,
    PorcentajeManganeso = 0.70
};

var aleacionReto2 = new Aleacion
{
    Nombre = "AISI 4140",
   Codigo = "A4140",
    PorcentajeCarbono = 0.40,
    PorcentajeManganeso = 0.90
};
```

y añade dos asignaciones:

```
ordenReto.OrdenesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacionReto1,
    CantidadUtilizada = 1000.250m,
    EstadoRelacion = "Activa"
});

ordenReto.OrdenesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacionReto2,
    CantidadUtilizada = 500.750m,
    EstadoRelacion = "Activa"
});
```

Después recupera:

```
var retoCargada = context.OrdenesFabricacion
    .AsNoTracking()
    .Include(o => o.OrdenesAleaciones)
    .ThenInclude(x => x.Aleacion)
    .Single(o => o.NumeroOrden == "OF-M2-RET0-25");
```

y debe mostrar:

```
Reto 2.5 aleaciones: 2
A1018 | Cantidad: 1000.250
A4140 | Cantidad: 500.750
```

La PK compuesta permite ambas filas porque cambia `AleacionId`; lo que no permitiría sería repetir dos veces el mismo par orden/aleación.

Paso 16: Verificar EstadoRelacion como dato propio de la relación

`EstadoRelacion` está mapeada en la entidad, forma parte de la migración y posee reglas propias:

```
entity.Property(x => x.EstadoRelacion)
    .IsRequired()
    .HasMaxLength(20)
    .HasDefaultValue("Activa");
```

En el código del laboratorio las relaciones se crean explícitamente como `"Activa"`. En SQL Server también existe el valor por defecto para inserciones que no proporcionen un valor desde el cliente.

El punto importante es conceptual: cuando la relación necesita fecha, cantidad, estado u otros atributos, deja de ser una simple asociación invisible y la entidad intermedia explícita se convierte en parte del dominio persistente.

Errores comunes

Error	Causa	Corrección
Se añade un <code>Id</code> a <code>OrdenAleacion</code>	Se trata la entidad puente como una entidad con PK simple	Mantener la PK compuesta <code>OrdenFabricacionId + AleacionId</code>
Se usan colecciones directas además de <code>OrdenesAleaciones</code>	Se modelan dos relaciones diferentes sin intención	Usar las colecciones de la entidad intermedia
Se espera un <code>DropTable</code> de una tabla implícita	Se supone que 2.4 ya tenía una relación M:N	La migración real de 2.5 crea directamente <code>OrdenesAleaciones</code>
Se omite <code>ThenInclude</code>	Sólo se carga la entidad intermedia	Añadir <code>.ThenInclude(x => x.Aleacion)</code>

Error	Causa	Corrección
Se repite el mismo par orden/aleación	La PK compuesta ya existe	No insertar una relación duplicada; actualizar la existente
Se configura <code>Cascade</code> en ambos lados	Se relaja la protección de la aleación compartida	Mantener <code>Cascade</code> hacia orden y <code>Restrict</code> hacia aleación
Se usa <code>EnsureCreated()</code>	Se evita el historial de migraciones	Aplicar Migrations
Se ignoran <code>CantidadUtilizada</code> o <code>EstadoRelacion</code>	Se pierde el motivo de usar entidad explícita	Tratar los datos de la relación como parte del modelo

Analugía final

`OrdenAleacion` se parece al libro de recetas de la acería. La orden indica qué se fabrica y la aleación describe un material disponible, pero la hoja de asignación registra **qué aleación se utilizó en qué orden, cuándo, en qué cantidad y con qué estado**. La combinación orden + aleación identifica de forma natural cada registro. Si se elimina la orden, sus asignaciones dejan de tener sentido; si una aleación sigue en uso, no debería eliminarse del catálogo.

Resultado esperado

Al finalizar 2.5, `AceriaData` conserva todo lo construido en 2.4 y añade la entidad `OrdenAleacion`, la tabla `OrdenesAleaciones`, una PK compuesta, las dos FKs con comportamientos de borrado diferentes y los datos propios `FechaAsignacion`, `CantidadUtilizada` y `EstadoRelacion`. La ejecución termina con `2.5 OK | Aleaciones: 1 | Primera: A1045`, y el reto demuestra dos aleaciones distintas con cantidades diferentes cargadas mediante `Include` y `ThenInclude`.

Conexión con el siguiente punto

El siguiente estado, 2.6, continúa desde esta solución y se centra en **Data Annotations**. La PK compuesta y la entidad intermedia creadas aquí permanecen en el modelo acumulativo.

Código acumulativo completo del estado 2.5

El siguiente archivo es el `Program.cs` real del estado ejecutable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}
```

```

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    public string? Notas { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class CertificadoCalidad
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    public string OrganismoCertificador { get; set; } = string.Empty;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    public decimal CantidadUtilizada { get; set; }
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();
}

```

```

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
    modelBuilder.Entity<OrdenFabricacion>(entity =>
    {
        entity.ToTable("OrdenesFabricacion");
        entity.HasKey(o => o.Id);
        entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
        entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
        entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
        entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
        entity.Property(o => o.Observaciones).HasMaxLength(500);
    });
    modelBuilder.Entity<PlanchaAcero>(entity =>
    {
        entity.ToTable("PlanchasAcero");
        entity.HasKey(x => x.Id);
        entity.Property(x => x.Peso).HasPrecision(18, 3);
        entity.Property(x => x.Activa).HasDefaultValue(true);
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.Planchas)
            .HasForeignKey(x => x.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<Aleacion>(entity =>
    {
        entity.ToTable("Aleaciones");
        entity.HasKey(a => a.Id);
        entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
        entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
        entity.Property(a => a.Descripcion).HasMaxLength(500);
    });
    modelBuilder.Entity<EstadoOrden>(entity =>
    {
        entity.ToTable("EstadosOrden");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
        entity.Property(e => e.Descripcion).HasMaxLength(250);
        entity.Property(e => e.Activo).HasDefaultValue(true);
    });
    modelBuilder.Entity<DetalleOrden>(entity =>
    {
        entity.ToTable("DetallesOrden");
        entity.HasKey(d => d.Id);
        entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
        entity.Property(d => d.Notas).HasMaxLength(500);
        entity.HasOne(d => d.Orden)
            .WithOne(o => o.Detalle)
            .HasForeignKey<DetalleOrden>(d => d.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<CertificadoCalidad>(entity =>
    {
        entity.ToTable("CertificadosCalidad");
        entity.HasKey(c => c.Id);
        entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
        entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
        entity.HasOne(c => c.Orden)
            .WithOne(o => o.Certificado)
            .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<OrdenAleacion>(entity =>
    {
        entity.ToTable("OrdenesAleaciones");
        entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
        entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
    });
}

```

```

        entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
        entity.Property(x =>
x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.OrdenesAleaciones)
            .HasForeignKey(x => x.OrdenFabricacionId)
            .OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(x => x.Aleacion)
            .WithMany(a => a.OrdenesAleaciones)
            .HasForeignKey(x => x.AleacionId)
            .OnDelete(DeleteBehavior.Restrict);
    });
}
}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o =>
o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
_context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOordenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOordenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();
    }
}

```

```

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

var services = new ServiceCollection();
services.AddDbContext<AceriaDbContext>(options =>
    options
        .UseSqlServer(connectionString, sql =>
            {
                sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                sql.CommandTimeout(60);
            })
        .LogTo(global::System.Console.WriteLine, new[] {
"Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)
        .EnableDetailedErrors());

services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "0F-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5,
        Notas = "Colada principal"
    },
    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OrdenesAleaciones.Add(new OrdenAleacion
{

```

```

        Aleacion = aleacion,
        CantidadUtilizada = 1500.500m,
        EstadoRelacion = "Activa"
    });
    context.OrdenesFabricacion.Add(orden);
    context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa",
Activo = true });
    context.SaveChanges();

    var cargada = context.OrdenesFabricacion.Include(o => o.OrdenesAleaciones).ThenInclude(x =>
x.Aleacion).Single(o => o.NumeroOrden == "OF-M2-0001");
    global::System.Console.WriteLine($"2.5 OK | Aleaciones: {cargada.OrdenesAleaciones.Count} |
Primera: {cargada.OrdenesAleaciones[0].Aleacion.Codigo}");
    }
}

```

Explicación línea a línea del código acumulativo 2.5

Las referencias siguientes usan la numeración real de M02/PROYECTO/2.5/Program.cs. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1: `using System.ComponentModel.DataAnnotations;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 2: `using System.ComponentModel.DataAnnotations.Schema;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 4: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 6: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 8: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres del estado acumulativo.

Línea 10: `public class OrdenFabricacion` → declara la clase OrdenFabricacion que forma parte del estado acumulativo del ejercicio.

Línea 11: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 12: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 13: `public string NumeroOrden { get; set; } = string.Empty;` → declara la propiedad NumeroOrden de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 14: `public string Cliente { get; set; } = string.Empty;` → declara la propiedad Cliente de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 15: `public DateTime FechaCreacion { get; set; }` → declara la propiedad FechaCreacion de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.

Línea 16: `public DateTime? FechaEntrega { get; set; }` → declara la propiedad FechaEntrega de tipo DateTime? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 17: `public string Estado { get; set; } = "Pendiente";` → declara la propiedad Estado de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 18: `public string? Observaciones { get; set; }` → declara la propiedad Observaciones de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 19: `public List<PlanchaAcero> Planchas { get; set; } = new();` → declara la propiedad Planchas de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 20: `public DetalleOrden? Detalle { get; set; }` → declara la propiedad Detalle de tipo DetalleOrden? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 21: `public CertificadoCalidad? Certificado { get; set; }` → declara la propiedad Certificado de tipo CertificadoCalidad? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 22: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 23: `}` → cierra el bloque de código actual.

Línea 25: `public class PlanchaAcero` → declara la clase PlanchaAcero que forma parte del estado acumulativo del ejercicio.

Línea 26: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 27: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 28: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 29: `public double Espesor { get; set; }` → declara la propiedad Espesor de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 30: `public double Ancho { get; set; }` → declara la propiedad Ancho de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 31: `public double Largo { get; set; }` → declara la propiedad Largo de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 32: `public decimal Peso { get; set; }` → declara la propiedad Peso de tipo decimal que EF Core o la lógica de negocio utiliza en este modelo.

Línea 33: `public bool Activa { get; set; } = true;` → declara la propiedad Activa de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.

Línea 34: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 35: `}` → cierra el bloque de código actual.

Línea 37: `public class Aleacion` → declara la clase Aleacion que forma parte del estado acumulativo del ejercicio.

Línea 38: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 39: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 40: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 41: `public string Codigo { get; set; } = string.Empty;` → declara la propiedad Codigo de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 42: `public double PorcentajeCarbono { get; set; }` → declara la propiedad PorcentajeCarbono de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 43: `public double PorcentajeManganeso { get; set; }` → declara la propiedad PorcentajeManganeso de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 44: `public string? Descripcion { get; set; }` → declara la propiedad Descripcion de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 45: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad `OrdenesAleaciones` de tipo `List` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 46: `}` → cierra el bloque de código actual.

Línea 48: `public class EstadoOrden` → declara la clase `EstadoOrden` que forma parte del estado acumulativo del ejercicio.

Línea 49: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 50: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 51: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad `Nombre` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 52: `public string Descripcion { get; set; } = string.Empty;` → declara la propiedad `Descripcion` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 53: `public bool Activo { get; set; } = true;` → declara la propiedad `Activo` de tipo `bool` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 54: `}` → cierra el bloque de código actual.

Línea 56: `public class DetalleOrden` → declara la clase `DetalleOrden` que forma parte del estado acumulativo del ejercicio.

Línea 57: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 58: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 59: `public int OrdenId { get; set; }` → declara la propiedad `OrdenId` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 60: `public string ComposicionQuimica { get; set; } = string.Empty;` → declara la propiedad `ComposicionQuimica` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 61: `public double TemperaturaColada { get; set; }` → declara la propiedad `TemperaturaColada` de tipo `double` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 62: `public string? Notas { get; set; }` → declara la propiedad `Notas` de tipo `string?` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 63: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 64: `}` → cierra el bloque de código actual.

Línea 66: `public class CertificadoCalidad` → declara la clase `CertificadoCalidad` que forma parte del estado acumulativo del ejercicio.

Línea 67: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 68: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 69: `public int OrdenId { get; set; }` → declara la propiedad `OrdenId` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 70: `public string NumeroCertificado { get; set; } = string.Empty;` → declara la propiedad `NumeroCertificado` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 71: `public DateTime FechaEmision { get; set; }` → declara la propiedad `FechaEmision` de tipo `DateTime` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 72: `public string OrganismoCertificador { get; set; } = string.Empty;` → declara la propiedad `OrganismoCertificador` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 73: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 74: `}` → cierra el bloque de código actual.

Línea 76: `public class OrdenAleacion` → declara la clase `OrdenAleacion` que forma parte del estado acumulativo del ejercicio.

Línea 77: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 78: `public int OrdenFabricacionId { get; set; }` → declara la propiedad `OrdenFabricacionId` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 79: `public int AleacionId { get; set; }` → declara la propiedad `AleacionId` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 80: `public DateTime FechaAsignacion { get; set; } = DateTime.Now;` → declara la propiedad `FechaAsignacion` de tipo `DateTime` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 81: `public decimal CantidadUtilizada { get; set; }` → declara la propiedad `CantidadUtilizada` de tipo `decimal` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 82: `public string EstadoRelacion { get; set; } = "Activa";` → declara la propiedad `EstadoRelacion` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 83: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 84: `public Aleacion Aleacion { get; set; } = null!;` → declara la propiedad `Aleacion` de tipo `Aleacion` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 85: `}` → cierra el bloque de código actual.

Línea 87: `public class AceriaDbContext : DbContext` → declara la clase `AceriaDbContext` que forma parte del estado acumulativo del ejercicio.

Línea 88: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 89: `public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → declara el constructor o método `AceriaDbContext` con los parámetros necesarios para este componente.

Línea 91: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 92: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 93: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 94: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 95: `public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 96: `public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 97: `public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 100: `protected override void OnModelCreating(ModelBuilder modelBuilder)` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 101: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 102: `base.OnModelCreating(modelBuilder);` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 103: `modelBuilder.Entity<OrdenFabricacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 104: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 105: `entity.ToTable("OrdenesFabricacion");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 106: `entity.HasKey(o => o.Id);` → define la clave primaria de la entidad.

Línea 107: `entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 108: `entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 109: `entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 110: `entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");` → selecciona una propiedad escalar para continuar su configuración.

Línea 111: `entity.Property(o => o.Observaciones).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 112: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 113: `modelBuilder.Entity<PlanchaAcero>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 114: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 115: `entity.ToTable("PlanchasAcero");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 116: `entity.HasKey(x => x.Id);` → define la clave primaria de la entidad.

Línea 117: `entity.Property(x => x.Peso).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 118: `entity.Property(x => x.Activa).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 119: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 120: `.WithMany(o => o.Planchas)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 121: `.HasForeignKey(x => x.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 122: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 123: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 124: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 125: `modelBuilder.Entity<Aleacion>(entity => → selecciona una entidad del modelo para aplicar su configuración Fluent API.`

Línea 126: `{ → abre el bloque de código asociado a la declaración o instrucción anterior.`

Línea 127: `entity.ToTable("Aleaciones"); → asigna explícitamente la tabla de SQL Server usada por la entidad.`

Línea 128: `entity.HasKey(a => a.Id); → define la clave primaria de la entidad.`

Línea 129: `entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100); → selecciona una propiedad escalar para continuar su configuración.`

Línea 130: `entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20); → selecciona una propiedad escalar para continuar su configuración.`

Línea 131: `entity.Property(a => a.Descripcion).HasMaxLength(500); → selecciona una propiedad escalar para continuar su configuración.`

Línea 132: `}); → cierra la lambda o configuración encadenada y finaliza la llamada.`

Línea 133: `modelBuilder.Entity<EstadoOrden>(entity => → selecciona una entidad del modelo para aplicar su configuración Fluent API.`

Línea 134: `{ → abre el bloque de código asociado a la declaración o instrucción anterior.`

Línea 135: `entity.ToTable("EstadosOrden"); → asigna explícitamente la tabla de SQL Server usada por la entidad.`

Línea 136: `entity.HasKey(e => e.Id); → define la clave primaria de la entidad.`

Línea 137: `entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50); → selecciona una propiedad escalar para continuar su configuración.`

Línea 138: `entity.Property(e => e.Descripcion).HasMaxLength(250); → selecciona una propiedad escalar para continuar su configuración.`

Línea 139: `entity.Property(e => e.Activo).HasDefaultValue(true); → selecciona una propiedad escalar para continuar su configuración.`

Línea 140: `}); → cierra la lambda o configuración encadenada y finaliza la llamada.`

Línea 141: `modelBuilder.Entity<DetalleOrden>(entity => → selecciona una entidad del modelo para aplicar su configuración Fluent API.`

Línea 142: `{ → abre el bloque de código asociado a la declaración o instrucción anterior.`

Línea 143: `entity.ToTable("DetallesOrden"); → asigna explícitamente la tabla de SQL Server usada por la entidad.`

Línea 144: `entity.HasKey(d => d.Id); → define la clave primaria de la entidad.`

Línea 145: `entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200); → selecciona una propiedad escalar para continuar su configuración.`

Línea 146: `entity.Property(d => d.Notas).HasMaxLength(500); → selecciona una propiedad escalar para continuar su configuración.`

Línea 147: `entity.HasOne(d => d.Orden) → inicia la configuración de una navegación de referencia.`

Línea 148: `.WithOne(o => o.Detalle) → establece el otro extremo de referencia de una relación uno-a-uno.`

Línea 149: `.HasForeignKey<DetalleOrden>(d => d.OrdenId) → declara explícitamente la propiedad o propiedades de clave foránea.`

Línea 150: `.OnDelete(DeleteBehavior.Cascade) → define el comportamiento de eliminación de la relación.`

Línea 151: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 152: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 154: `modelBuilder.Entity<CertificadoCalidad>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 155: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 156: `entity.ToTable("CertificadosCalidad");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 157: `entity.HasKey(c => c.Id);` → define la clave primaria de la entidad.

Línea 158: `entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 159: `entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 160: `entity.HasOne(c => c.Orden)` → inicia la configuración de una navegación de referencia.

Línea 161: `.WithOne(o => o.Certificado)` → establece el otro extremo de referencia de una relación uno-a-uno.

Línea 162: `.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 163: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 164: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 165: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 166: `modelBuilder.Entity<OrdenAleacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 167: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 168: `entity.ToTable("OrdenesAleaciones");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 169: `entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });` → define la clave primaria de la entidad.

Línea 170: `entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 171: `entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 172: `entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");` → selecciona una propiedad escalar para continuar su configuración.

Línea 173: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 174: `.WithMany(o => o.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 175: `.HasForeignKey(x => x.OrdenFabricacionId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 176: `.OnDelete(DeleteBehavior.Cascade);` → define el comportamiento de eliminación de la relación.

Línea 177: `entity.HasOne(x => x.Aleacion)` → inicia la configuración de una navegación de referencia.

Línea 178: `.WithMany(a => a.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 179: `.HasForeignKey(x => x.AleacionId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 180: `.OnDelete(DeleteBehavior.Restrict);` → define el comportamiento de eliminación de la relación.

Línea 181: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 182: `}` → cierra el bloque de código actual.

Línea 183: `}` → cierra el bloque de código actual.

Línea 185: `public interface IOrdenRepositorio` → declara el contrato IOrdenRepositorio para desacoplar consumidores e implementación.

Línea 186: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 187: `List<OrdenFabricacion> ObtenerTodas();` → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 188: `OrdenFabricacion? ObtenerPorId(int id);` → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 189: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.

Línea 190: `void Agregar(OrdenFabricacion orden);` → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.

Línea 191: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.

Línea 192: `int Guardar();` → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 193: `}` → cierra el bloque de código actual.

Línea 195: `public sealed class OrdenRepositorio : IOrdenRepositorio` → declara la clase OrdenRepositorio que forma parte del estado acumulativo del ejercicio.

Línea 196: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 197: `private readonly AceriaDbContext _context;` → declara el campo privado _context de tipo AceriaDbContext que conserva una dependencia de la clase.

Línea 198: `public OrdenRepositorio(AceriaDbContext context) => _context = context;` → declara el constructor o método OrdenRepositorio con los parámetros necesarios para este componente.

Línea 200: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → ordena el resultado de la consulta antes de materializarlo.

Línea 201: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave primaria mediante Find, aprovechando primero el ChangeTracker y después la base de datos.

Línea 202: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → recupera la primera coincidencia o null cuando no existe.

Línea 203: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.

Línea 204: `public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);` → marca la entidad indicada para borrado al confirmar los cambios.

Línea 205: `public int Guardar() => _context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el DbContext.

Línea 206: `}` → cierra el bloque de código actual.

Línea 208: `public interface IServicioOrdenes` → declara el contrato IServicioOrdenes para desacoplar consumidores e implementación.

Línea 209: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 210: `string ObtenerResumen(int ordenId);` → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 211: `}` → cierra el bloque de código actual.

Línea 213: `public sealed class ServicioOrdenes : IServicioOrdenes` → declara la clase ServicioOrdenes que forma parte del estado acumulativo del ejercicio.

Línea 214: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 215: `private readonly IOrdenRepositorio _repositorio;` → declara el campo privado _repositorio de tipo IOrdenRepositorio que conserva una dependencia de la clase.

Línea 216: `private readonly AceriaDbContext _context;` → declara el campo privado _context de tipo AceriaDbContext que conserva una dependencia de la clase.

Línea 217: `public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)` → declara el constructor o método ServicioOrdenes con los parámetros necesarios para este componente.

Línea 218: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 219: `_repositorio = repositorio;` → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.

Línea 220: `_context = context;` → guarda el DbContext inyectado para consultar los datos necesarios durante el servicio.

Línea 221: `}` → cierra el bloque de código actual.

Línea 223: `public string ObtenerResumen(int ordenId)` → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.

Línea 224: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 225: `var orden = _repositorio.ObtenerPorId(ordenId);` → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.

Línea 226: `if (orden is null) return $"Orden {ordenId} no encontrada";` → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.

Línea 227: `var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);` → cuenta las filas que cumplen la consulta construida.

Línea 228: `return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";` → devuelve el resultado calculado al código que invocó el método.

Línea 229: `}` → cierra el bloque de código actual.

Línea 230: `}` → cierra el bloque de código actual.

Línea 232: `public static class Program` → declara la clase Program que forma parte del estado acumulativo del ejercicio.

Línea 233: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 234: `public static void Main()` → declara el punto de entrada ejecutable de la aplicación de consola.

Línea 235: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 236: `var configuration = new ConfigurationBuilder()` → inicia la construcción de la configuración de la aplicación.

Línea 237: `.SetBasePath(AppContext.BaseDirectory)` → establece la ruta base usada para localizar los archivos de configuración.

Línea 238: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)` → incorpora appsettings.json a la configuración.

Línea 239: `.AddEnvironmentVariables()` → incorpora variables de entorno como fuente adicional de configuración.

Línea 240: `.Build();` → materializa la configuración o el objeto encadenado construido en las líneas anteriores.

Línea 242: `var connectionString = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena de conexión AceriaDB desde la configuración.

Línea 243: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.

Línea 245: `var services = new ServiceCollection();` → crea la colección donde se registran las dependencias de la aplicación.

Línea 246: `services.AddDbContext<AceriaDbContext>(options =>` → registra el DbContext en el contenedor de inyección de dependencias.

Línea 247: `options` → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de AddDbContext.

Línea 248: `.UseSqlServer(connectionString, sql =>` → selecciona SQL Server como proveedor de EF Core.

Línea 249: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 250: `sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);` → habilita reintentos ante fallos transitorios del proveedor SQL Server.

Línea 251: `sql.CommandTimeout(60);` → establece el tiempo máximo permitido para los comandos SQL.

Línea 252: `})` → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.

Línea 253: `.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)` → configura la salida del logging de EF Core.

Línea 254: `.EnableDetailedErrors();` → activa errores detallados para facilitar el diagnóstico del laboratorio.

Línea 256: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();` → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 257: `services.AddScoped<IServicioOrdenes, ServicioOrdenes>();` → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 259: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → construye el ServiceProvider con validación de servicios y garantiza su liberación al salir del ámbito mediante using var.

Línea 260: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 261: `ValidateScopes = true,` → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.

Línea 262: `ValidateOnBuild = true` → obliga al contenedor a validar las resoluciones registradas al construir el `ServiceProvider`.

Línea 263: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 265: `using var scope = provider.CreateScope();` → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.

Línea 266: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → resuelve un servicio obligatorio desde el contenedor de dependencias.

Línea 267: `context.Database.EnsureDeleted();` → elimina la base del laboratorio para reproducir el escenario desde un estado limpio.

Línea 268: `context.Database.Migrate();` → aplica las migraciones pendientes y deja `AceriaDB` en el esquema del punto.

Línea 270: `var orden = new OrdenFabricacion` → crea la entidad `OrdenFabricacion` que concentra el grafo de datos del escenario acumulativo.

Línea 271: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 272: `NumeroOrden = "0F-M2-0001",` → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.

Línea 273: `Cliente = "Constructora del Norte",` → asigna el cliente de la orden utilizada en el escenario práctico del punto.

Línea 274: `FechaCreacion = DateTime.UtcNow,` → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.

Línea 275: `FechaEntrega = DateTime.Today.AddDays(14),` → establece una fecha de entrega prevista catorce días después de la fecha local actual.

Línea 276: `Estado = "Pendiente",` → inicializa la orden con el estado `Pendiente`, coherente con el flujo del laboratorio.

Línea 277: `Observaciones = "Orden de validación M2",` → añade texto opcional para comprobar el mapeo de `Observaciones`.

Línea 278: `Planchas =` → inicia el inicializador de la colección de planchas asociadas a la orden.

Línea 279: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 280: `new PlanchaAcero` → crea una nueva plancha dentro del inicializador de la colección `Planchas` de la orden.

Línea 281: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 282: `Espesor = 10.5,` → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.

Línea 283: `Ancho = 1500,` → asigna el ancho de la plancha usada en el escenario de persistencia.

Línea 284: `Largo = 3000,` → asigna el largo de la plancha usada en el escenario de persistencia.

Línea 285: `Peso = 371.250m,` → asigna un peso decimal con tres posiciones para validar la precisión configurada.

Línea 286: `Activa = true,` → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.

Línea 287: `}` → cierra el bloque de código actual.

Línea 288: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 289: `Detalle = new DetalleOrden` → crea el detalle uno-a-uno que se persistirá junto con la orden principal.

Línea 290: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 291: `ComposicionQuimica = "C: 0.45%, Mn: 0.75%",` → asigna la composición química utilizada para verificar la entidad `DetalleOrden`.

Línea 292: `TemperaturaColada = 1550.5,` → asigna la temperatura de colada del detalle de fabricación.

Línea 293: `Notas = "Colada principal"` → asigna las notas opcionales del detalle asociado a la orden.

Línea 294: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 295: `Certificado = new CertificadoCalidad` → crea el certificado uno-a-uno asociado a la orden del escenario.

Línea 296: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 297: `NumeroCertificado = "CERT-0001",` → asigna el identificador natural del certificado de calidad.

Línea 298: `FechaEmision = DateTime.Today,` → asigna la fecha de emisión del certificado usado en la prueba.

Línea 299: `OrganismoCertificador = "Laboratorio Aceria"` → asigna el organismo que emite el certificado de calidad.

Línea 300: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 301: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 303: `var aleacion = new Aleacion` → crea la aleación que se enlazarán con la orden mediante la entidad intermedia `OrdenAleacion`.

Línea 304: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 305: `Nombre = "AISI 1045",` → asigna el nombre comercial de la aleación utilizada en el escenario.

Línea 306: `Codigo = "A1045",` → asigna el código natural único de la aleación.

Línea 307: `PorcentajeCarbono = 0.45,` → asigna el porcentaje de carbono dentro del rango permitido por la restricción del modelo.

Línea 308: `PorcentajeManganeso = 0.75,` → asigna el porcentaje de manganeso dentro del rango permitido por la restricción del modelo.

Línea 309: `Descripcion = "Acero medio en carbono"` → asigna una descripción opcional a la aleación.

Línea 310: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 311: `orden.OrdenesAleaciones.Add(new OrdenAleacion` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 312: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 313: `Aleacion = aleacion,` → conecta la entidad intermedia `OrdenAleacion` con la aleación creada para el escenario.

Línea 314: `CantidadUtilizada = 1500.500m,` → registra la cantidad utilizada con tres decimales, coherente con `HasPrecision(18, 3)`.

Línea 315: `EstadoRelacion = "Activa"` → marca como activa la relación entre la orden y la aleación.

Línea 316: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 317: `context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 318: `context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 319: `context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el `DbContext`.

Línea 321: `var cargada = context.OrdenesFabricacion.Include(o => o.OrdenesAleaciones).ThenInclude(x => x.Aleacion).Single(o => o.NumeroOrden == "OF-M2-0001");` → incluye de forma eager la navegación relacionada en la consulta.

Línea 322: `global::System.Console.WriteLine($"2.5 OK | Aleaciones: {cargada.OrdenesAleaciones.Count} | Primera: {cargada.OrdenesAleaciones[0].Aleacion.Codigo}");` → escribe en consola la evidencia usada para verificar el comportamiento del estado.

Línea 323: `}` → cierra el bloque de código actual.

Línea 324: `}` → cierra el bloque de código actual.